DevOps — Jira / ITSM + Production Support + GitLab CI/CD
3–4 YOE Scenario-Based Interview Q&A

Jira / ITSM Workflow

1. Production lo application down ayindi. Jira ticket vachinappudu first em chesthav?
Answer:
First, I check the ticket details, affected application, environment, time of occurrence, severity, and business impact. Then I acknowledge the ticket and start troubleshooting using application, server, monitoring, and deployment information.
I document my findings and actions in the ticket so that other teams can understand what has already been checked. After resolving the issue, I validate the application and update the ticket with the resolution and evidence.
Simple ga gurthupettuko:
Production issue → impact check → acknowledge → troubleshoot → update ticket → fix → verify → close.
Eng: First I understand the impact and severity, troubleshoot the issue, document my actions, verify the fix, and update the Jira ticket before closure.
Example:
Jira → Incident ticket open chesi → application, environment, impact, timestamp details chudu.
Monitoring → application status/alerts check cheyyi.
kubectl get pods -n <namespace> → application Pods healthy ga unnaya chudu.
kubectl logs <pod> -n <namespace> → application errors check cheyyi.
Issue identify chesi → fix apply cheyyi → application health verify cheyyi.
Jira ticket → troubleshooting steps + root cause + resolution + validation evidence update chesi → ticket close cheyyi.

---

2. Jira lo P1 incident vachindi. P1 ante ela handle chesthav?
Answer:
P1 means a critical incident with major business or Production impact, so I immediately acknowledge it and follow the organization's incident-management process. I quickly identify the impact, involve the required teams, and start troubleshooting without waiting for normal ticket timelines.
I provide regular status updates to stakeholders and focus first on restoring service. After recovery, I document the root cause and corrective actions.
Simple ga gurthupettuko:
P1 → immediate response → right teams involve → service restore → regular updates → RCA.
Eng: For a P1, I prioritize immediate service restoration, involve the required teams, communicate regularly, and complete RCA afterward.
Example:
Jira → Priority = P1 → affected service and business impact identify cheyyi.
Monitoring → active alerts and affected instances check cheyyi.
kubectl get pods -A → widespread Kubernetes issue undha chudu.
systemctl status <service> → server-side service status check cheyyi.
Relevant team ni incident bridge/call lo involve cheyyi.
Service restore ayyaka → Jira lo timeline, actions, impact, resolution update cheyyi.

---

3. P2 incident and P1 incident madhya difference enti?
Answer:
I mainly differentiate them based on business impact, number of affected users or services, and urgency. A P1 generally represents critical or widespread Production impact, while P2 is high impact but usually less critical or more contained.
The exact definitions depend on the organization's incident policy, so I follow the company's severity matrix and SLA.
Simple ga gurthupettuko:
P1 = critical/highest impact.
P2 = high impact but comparatively less critical.
Company severity matrix final.
Eng: I classify incidents based on business impact and follow the organization's documented severity and SLA definitions.
Example:
Jira → Incident priority/severity matrix check cheyyi.
Affected service/users count identify cheyyi.
Production impact → P1/P2 criteria tho compare cheyyi.
Correct priority set chesi → required response/escalation process follow cheyyi.

---

4. Jira ticket lo SLA ante enti?
Answer:
SLA, or Service Level Agreement, defines the expected response and resolution time for a particular ticket or service. I monitor the SLA timer and prioritize tickets that are approaching breach, especially critical incidents.
If resolution requires another team, I escalate early rather than waiting until the SLA is already breached.
Simple ga gurthupettuko:
SLA = ticket ni respond/resolve cheyyalsina agreed time.
Eng: SLA defines the expected response and resolution timelines, and I monitor tickets to avoid SLA breaches.
Example:
Jira → SLA field/timer check cheyyi.
Remaining response/resolution time chudu.
Ticket unresolved and SLA near breach ayithe → priority reassess chesi → relevant team ki escalate cheyyi.
Jira comments lo escalation and current status update cheyyi.

5. Jira ticket lo issue resolve cheyyalekapothe em chesthav?
Answer:
I first document what I checked, the observed error, logs, commands, and current findings. Then I identify the correct resolver team based on the issue—for example, Network, Security, Database, Development, or Cloud team.
I escalate with useful technical evidence instead of simply saying that the issue is not mine.
Simple ga gurthupettuko:
Solve avvakapothe → evidence collect → correct team identify → technical details tho escalate.
Eng: I escalate with complete technical evidence and clearly explain what I already checked.
Example:
Jira → troubleshooting steps and errors update cheyyi.
curl / kubectl / journalctl / application logs → relevant evidence collect cheyyi.
Issue network-related ayithe → Network team ki assign/escalate cheyyi.
Jira → error, timestamps, affected host, commands/results, and impact mention cheyyi.

---

6. Jira incident ticket ki technical update ela rayali?
Answer:
I keep the update clear and factual. I mention the current impact, what I checked, what I found, what action I took, and the current status.
I avoid vague comments such as "checking" without useful information.
Simple ga gurthupettuko:
Impact → checked → finding → action → status.
Eng: I provide concise updates covering impact, investigation, findings, actions, and current status.
Example:
Jira comment: "Production API returning 502. Checked Nginx logs and found upstream connection failure. Backend service was restarted and health endpoint is now responding successfully. Monitoring is being observed for stability."
curl -v http://<backend>:<port>/health → backend health verify cheyyi.
tail -100 /var/log/nginx/error.log → error evidence check cheyyi.

---

7. Same Production issue repeatedly vastundi. Jira lo em chesthav?
Answer:
If the same incident keeps recurring, I don't treat every occurrence as an isolated incident. I link the incidents and raise or associate a Problem record for root-cause investigation.
The goal is to identify the underlying cause and implement a permanent corrective action.
Simple ga gurthupettuko:
Same issue repeat → incidents link → Problem ticket → RCA → permanent fix.
Eng: Repeated incidents should lead to problem management and a permanent root-cause fix.
Example:
Jira → previous related incidents search cheyyi.
Repeated tickets ni Problem record tho link cheyyi.
Logs/monitoring history compare cheyyi.
Common root cause identify chesi → RCA document cheyyi.
Permanent fix implement chesi → future incidents reduce ayyaya monitor cheyyi.

---

8. Incident, Service Request, Change, Problem madhya difference enti?
Answer:
An Incident is an unplanned service interruption or degradation. A Service Request is a standard user or operational request, such as access or a routine action.
A Change is a planned modification to an environment or service, while a Problem focuses on identifying and eliminating the root cause of recurring incidents.
Simple ga gurthupettuko:
Incident = problem happened.
Request = something needed.
Change = planned modification.
Problem = root cause of recurring incidents.
Eng: Incident restores service, Request fulfills a need, Change modifies the environment, and Problem addresses recurring root causes.
Example:
"Production API down" → Incident.
"Give me read-only AWS access" → Service Request.
"Deploy new Nginx configuration tonight" → Change.
"Why does API fail every week?" → Problem.

---

9. Production deployment kosam Jira Change ticket enduku create chesthav?
Answer:
A Change ticket provides traceability and controlled approval for a planned Production modification. It records what is changing, why it is changing, when it will happen, the implementation plan, validation steps, and rollback plan.
This reduces the risk of uncontrolled Production changes.
Simple ga gurthupettuko:
Production change → why → what → when → implementation → validation → rollback → approval.
Eng: A Change ticket provides approval, traceability, implementation details, validation, and rollback information.

Example:
Jira → Change ticket create cheyyi.
Change reason → new application version mention cheyyi.
Implementation plan → deployment steps add cheyyi.
Validation → health check/test steps add cheyyi.
Rollback → previous image/version mention cheyyi.
Approval complete ayyaka → planned window lo deployment execute cheyyi.

---

10. Production issue resolve ayyaka Jira ticket ni ela close chesthav?
Answer:
Before closing, I verify that the service is healthy and the original problem is no longer occurring. Then I document the root cause, resolution, validation results, and any follow-up action.
If a permanent fix is still pending, I keep or link the appropriate Problem/Change ticket instead of incorrectly marking everything as permanently resolved.
Simple ga gurthupettuko:
Fix → verify → RCA/resolution update → follow-up link → close.
Eng: I close the incident only after validating recovery and documenting the resolution and follow-up actions.
Example:
curl -I https://<application> → HTTP response verify cheyyi.
kubectl get pods -n <namespace> → Pods healthy aa chudu.
Monitoring → alerts clear ayyaya check cheyyi.
Jira → root cause + resolution + validation evidence add cheyyi.
Pending permanent fix unte → Problem/Change ticket link chesi → Incident close cheyyi.

Production Support / On-Call
11. On-call lo alert vachindi. First 10–15 minutes ela handle chesthav?
Answer:
First, I acknowledge the alert and determine whether it is a real issue or a false positive. Then I check the impact, affected service, recent deployments or changes, and relevant logs and metrics.
For a critical Production issue, I immediately follow the escalation process while continuing the technical investigation.
Simple ga gurthupettuko:
Alert → acknowledge → real issue aa → impact → recent change → logs/metrics → escalate if required.
Eng: I quickly validate the alert, assess impact, investigate the likely cause, and escalate critical incidents early.
Example:
Monitoring → alert details and timestamp check cheyyi.
kubectl get pods -n <namespace> → workload status chudu.
kubectl top pods -n <namespace> → resource usage check cheyyi.
kubectl logs <pod> -n <namespace> --since=15m → recent errors chudu.
Recent deployment/change history check chesi → required team ki escalate cheyyi.

---

12. Incident bridge/call lo DevOps engineer ga nee role enti?
Answer:
My role is to provide technical investigation, execute approved recovery actions, and communicate clear technical status. I focus on facts rather than speculation and keep track of what has been tried and what the current impact is.
I also coordinate with application, database, network, security, or cloud teams when the issue crosses team boundaries.
Simple ga gurthupettuko:
Bridge lo → technical investigation + actions + status + coordination.
Eng: I focus on technical recovery, clear status updates, evidence, and coordination with other resolver teams.
Example:
Incident bridge → current impact and timeline communicate cheyyi.
Commands/results → relevant technical findings share cheyyi.
Application issue → Dev team involve cheyyi.
DB connectivity issue → DBA team involve cheyyi.
Every major action → incident timeline/Jira lo document cheyyi.

---

13. Stakeholder "When will the issue be fixed?" ani adigithe em chepthav?
Answer:
I avoid giving an unsupported ETA. I communicate the current impact, what has been identified, what action is in progress, and the next update time.
Once the root cause and recovery path are clear, I provide a realistic estimate.
Simple ga gurthupettuko:
Guess cheyyaku → current status cheppu → next update time cheppu → confirmed ETA only when known.
Eng: I communicate facts and realistic expectations rather than giving an unsupported ETA.
Example:
"Current impact is limited to Production API requests. We identified the failing backend and are validating the recovery. Next status update will be provided after the health checks complete."
Jira → same status update document cheyyi.

14. Incident handover vere engineer ki ela chesthav?
Answer:
I provide the current impact, timeline, symptoms, investigation already completed, findings, actions taken, current status, and next steps.
A good handover should allow the next engineer to continue without repeating the same troubleshooting.
Simple ga gurthupettuko:
Impact → timeline → checked → found → action → current status → next step.
Eng: I give a complete technical handover so the next engineer can continue from the current state.
Example:
Jira → complete timeline and troubleshooting notes update cheyyi.
"Checked Nginx → backend port reachable → application logs show DB timeout → DBA team engaged → monitoring ongoing."
Pending action and owner clearly mention cheyyi.

---

15. Production issue fix chesaka root cause analysis ela chesthav?
Answer:
I build a timeline of the incident, identify the triggering change or failure, and use logs, monitoring, deployment history, and configuration history as evidence.
I distinguish the immediate symptom from the underlying root cause and document corrective and preventive actions.
Simple ga gurthupettuko:
Timeline → evidence → trigger → root cause → corrective action → preventive action.
Eng: I use evidence from logs, monitoring, changes, and deployments to identify the root cause and prevent recurrence.
Example:
Jira → incident timeline collect cheyyi.
git log -10 --oneline → recent application changes chudu.
kubectl rollout history deployment/<name> -n <namespace> → deployment history check cheyyi.
Monitoring → incident time metrics compare cheyyi.
Root cause identify chesi → RCA + preventive action document cheyyi.

GitLab CI/CD
16. GitLab CI/CD ante enti?
Answer:
GitLab CI/CD is GitLab's built-in automation capability for building, testing, scanning, packaging, and deploying applications.
Pipeline configuration is commonly defined in a .gitlab-ci.yml file stored with the source code.
Jobs run through GitLab Runners, which execute the defined commands.
Simple ga gurthupettuko:
GitLab → .gitlab-ci.yml → pipeline/jobs → Runner → build/test/deploy.
Eng: GitLab CI/CD automates software delivery using pipeline definitions and GitLab Runners to execute jobs.
Example:
Repository root lo .gitlab-ci.yml → pipeline configuration chudu.
GitLab → CI/CD → Pipelines → pipeline status check cheyyi.
GitLab → CI/CD → Jobs → individual job logs open cheyyi.
Runner → actual job execution chestundi.

---

17. GitLab pipeline suddenly fail ayindi. Ela troubleshoot chesthav?
Answer:
First, I open the failed pipeline and identify the exact failed job and error from the job logs. Then I compare it with the last successful pipeline and check recent code, configuration, dependency, Runner, credential, or environment changes.
After fixing the root cause, I rerun the pipeline and verify all stages.
Simple ga gurthupettuko:
Failed job → logs → last successful compare → recent change → fix → rerun.
Eng: I identify the failed job, compare it with the last successful pipeline, fix the root cause, and rerun it.
Example:
GitLab → CI/CD → Pipelines → failed pipeline → failed Job → Logs open cheyyi.
GitLab → previous successful pipeline → logs compare cheyyi.
git log -5 --oneline → recent commits check cheyyi.
Runner status → Runner available aa chudu.
Issue fix chesi → Retry job → pipeline success verify cheyyi.

---

18. GitLab Runner is offline. What will you check?
Answer:
First, I check the Runner status in GitLab and identify which Runner the job requires. Then I check whether the Runner machine is running, the Runner service is healthy, network connectivity is available, and the Runner has the required tags.
After restoring the Runner, I retry the job.
Simple ga gurthupettuko:
Runner offline → status → machine/service → network → tags → retry.
Eng: I verify the Runner status, host, service, connectivity, and tags before retrying the job.
Example:
GitLab → Settings → CI/CD → Runners → Runner status chudu.
systemctl status gitlab-runner → Runner service check cheyyi.
journalctl -u gitlab-runner -n 100 --no-pager → Runner errors chudu.
gitlab-runner verify → Runner registration verify cheyyi.

Runner tags and .gitlab-ci.yml tags compare cheyyi.
Issue fix ayyaka → failed job retry cheyyi.

---

19. GitLab job is stuck in "Pending". What will you check?
Answer:
A pending job usually means GitLab has not found a suitable Runner to execute it. I check Runner availability, Runner tags, protected status, concurrency, and whether the Runner supports the required executor.
I correct the Runner assignment or availability issue and verify that the job starts.
Simple ga gurthupettuko:
Pending → suitable Runner dorikinda → tags → protected Runner → capacity → executor.
Eng: I check why GitLab cannot assign a suitable Runner to the pending job.
Example:
GitLab → Job → "This job is stuck..." message inspect cheyyi.
GitLab → CI/CD → Runners → available Runner check cheyyi.
.gitlab-ci.yml → tags verify cheyyi.
gitlab-runner list → configured Runners chudu.
Runner capacity/concurrent jobs check cheyyi.
Correct Runner available ayyaka → job execute ayyindo verify cheyyi.

---

20. GitLab pipeline works on one branch but fails on another. What will you compare?
Answer:
I compare the source-code changes, .gitlab-ci.yml, variables, branch rules, dependencies, and environment configuration between the two branches.
I also check whether the branch uses different Runner tags or protected variables.
Simple ga gurthupettuko:
Branch difference → code → CI config → variables → rules → Runner → dependencies.
Eng: I compare the branches for code, pipeline configuration, variables, rules, and Runner requirements.
Example:
git diff <working-branch>..<failing-branch> → code/config difference chudu.
git diff <working-branch>..<failing-branch> -- .gitlab-ci.yml → pipeline config compare cheyyi.
GitLab → Settings → CI/CD → Variables → protected/environment-specific variables check cheyyi.
Pipeline logs → actual difference identify cheyyi.
Required configuration fix chesi → pipeline rerun cheyyi.

---

21. GitLab pipeline lo secret/password expose ayindi. Em chesthav?
Answer:
I treat the exposed credential as compromised. I immediately rotate or revoke it, determine where it was exposed, remove the logging or configuration problem, and check whether the secret appeared in artifacts or other logs.
I then update the pipeline to use protected or masked CI/CD variables or a proper secret-management solution.
Simple ga gurthupettuko:
Secret exposed → rotate/revoke → exposure identify → clean → secure variable → verify.
Eng: I rotate the exposed secret immediately, remove the exposure, and move secret handling to secure CI/CD variables or secret management.
Example:
GitLab → Settings → CI/CD → Variables → secret variable identify cheyyi.
Exposed token/password → revoke/rotate cheyyi.
Job logs/artifacts → secret ekkada print ayyindo search cheyyi.
.gitlab-ci.yml → echo $SECRET type unsafe logging remove cheyyi.
Masked/Protected variable configure chesi → pipeline rerun → logs lo secret kanipinchadam ledu ani verify cheyyi.

---

22. GitLab pipeline lo Docker image build job fail ayindi. Ela troubleshoot chesthav?
Answer:
First, I check the job log and identify whether the failure is related to the Dockerfile, build context, authentication, Runner executor, Docker daemon, or registry access.
Then I validate the Dockerfile and check whether the Runner has the required Docker capability and resources.
Simple ga gurthupettuko:
Docker job fail → logs → Dockerfile → Runner → Docker daemon → registry → resources.
Eng: I identify whether the failure is in the Docker build, Runner environment, authentication, or registry access.
Example:
GitLab → Job → Logs → exact Docker error chudu.
docker build -t test-image . → Dockerfile locally/agent side test cheyyi.
docker info → Docker daemon availability check cheyyi.
df -h → disk space check cheyyi.
docker login <registry> → registry authentication test cheyyi.
Issue fix chesi → GitLab job retry → image build success verify cheyyi.

23. GitLab pipeline lo environment variable value correct ga ravatledu. What will you check?
Answer:
I check where the variable is defined, whether it is project-level, group-level, job-level, environment-specific, protected, or masked.
I also check variable precedence and whether the pipeline is running on a branch or environment where the variable is available.
Simple ga gurthupettuko:
Variable wrong → ekkada define → precedence → protected → environment → branch check.
Eng: I verify the variable source, precedence, protection settings, environment scope, and branch conditions.
Example:
GitLab → Settings → CI/CD → Variables → variable scope check cheyyi.
.gitlab-ci.yml → same variable job-level lo override ayyinda chudu.
Protected variable ayithe → protected branch/tag nunchi pipeline run avutunda verify cheyyi.
Job log lo echo $VAR badulu → safe/non-secret existence check use cheyyi.
Correct scope/config fix chesi → pipeline rerun cheyyi.

---

24. GitLab pipeline deploy job succeeds, but application is not updated. What will you check?
Answer:
I verify whether the job actually deployed the intended version and whether the correct environment and artifact/image were used.
Then I check deployment logs, target application status, image/version, and rollout status.
I also verify whether the deployment job reported success before the application became healthy.
Simple ga gurthupettuko:
Job success ≠ application success → version → environment → deployment → rollout → health check.
Eng: I verify the deployed version and target environment and then confirm the application's actual rollout and health.
Example:
GitLab → Deploy job logs → deployed image/version identify cheyyi.
kubectl get deployment <name> -n <namespace> -o yaml → image verify cheyyi.
kubectl rollout status deployment/<name> -n <namespace> → rollout check cheyyi.
kubectl get pods -n <namespace> → new Pods running aa chudu.
curl http://<service>/health → application health verify cheyyi.

---

25. GitLab CI/CD lo build, test, deploy stages ni ela structure chesthav?
Answer:
I separate the pipeline into clear stages such as build, test, security/quality checks, package, and deploy. Independent jobs can run in parallel, while deployment jobs should depend on successful validation stages.
Production deployment should also have appropriate approval and environment controls.
Simple ga gurthupettuko:
Build → Test → Quality/Security → Package → Deploy → Verify.
Eng: I structure the pipeline into clear stages and allow deployment only after the required validation stages succeed.
Example:
.gitlab-ci.yml → stages define cheyyi: build, test, scan, deploy.
Build job → artifact/image create cheyyi.
Test jobs → unit/integration tests run cheyyi.
Scan → quality/security validation run cheyyi.
Deploy → required environment ki deploy cheyyi.
Production → protected environment/manual approval configure chesi → deployment verify cheyyi.

---

26. GitLab CI/CD lo same pipeline logic multiple projects lo undi. Ela manage chesthav?
Answer:
I avoid copying large amounts of pipeline logic into every project. I use reusable CI/CD templates or centralized configuration where appropriate and keep project-specific values separate.
This reduces duplication and makes common changes easier to maintain.
Simple ga gurthupettuko:
Same CI code → copy-paste vaddu → reusable templates → project-specific config separate.
Eng: I use reusable CI/CD templates to reduce duplicated pipeline logic across projects.
Example:
Common build/test configuration → reusable GitLab CI template create cheyyi.
Project .gitlab-ci.yml → template include cheyyi.
Project-specific image name/environment variables separate ga maintain cheyyi.
Common template change chesi → affected projects pipelines verify cheyyi.

---

27. GitLab deployment job accidentally Production ki run kakunda ela prevent chesthav?
Answer:
I protect Production using protected branches/tags, protected environments, restricted deployment permissions, and manual approval where required.
I also use CI rules so that Production deployment is available only under approved conditions.
Simple ga gurthupettuko:
Production → protected branch/environment → restricted access → approval → deploy.
Eng: I protect Production through branch, environment, permission, and approval controls.
Example:
GitLab → Settings → Repository → Protected Branches → Production branch protect cheyyi.
GitLab → Settings → CI/CD → Protected Environments → Production access restrict cheyyi.

.gitlab-ci.yml → Production job ki appropriate rules/manual control add cheyyi.
Unauthorized branch nunchi pipeline run chesi → Production deploy available kadu ani verify cheyyi.

---

28. GitLab pipeline lo dependency download fail ayindi. What will you check?
Answer:
I check the dependency repository URL, network connectivity, DNS, credentials, proxy configuration, and whether the requested version exists.
I also compare the failing Runner with a working environment because the issue may be specific to the Runner network or configuration.
Simple ga gurthupettuko:
Dependency fail → repo → DNS/network → credentials → proxy → version → Runner compare.
Eng: I verify repository access, authentication, network configuration, and dependency availability.
Example:
curl -I <repository-url> → repository reachable aa check cheyyi.
nslookup <repository-host> → DNS verify cheyyi.
curl -u <user>:<token> -I <artifact-url> → authentication test cheyyi.
Dependency version repository lo exists aa chudu.
Runner environment/proxy settings compare chesi → issue fix → pipeline rerun cheyyi.

---

29. GitLab Runner lo disk full ayindi and jobs fail avutunnayi. Em chesthav?
Answer:
First, I identify what is consuming the Runner disk, such as old workspaces, Docker images, containers, caches, or artifacts.
I safely clean unnecessary data, verify free space, and configure appropriate cleanup policies so the problem does not repeatedly occur.
Simple ga gurthupettuko:
Runner disk full → space consumer → cleanup → verify → prevention.
Eng: I identify the disk consumer, safely clean unnecessary data, and implement cleanup controls.
Example:
df -h → disk usage check cheyyi.
du -sh /var/lib/docker/* → Docker storage inspect cheyyi.
docker system df → Docker disk usage chudu.
Unused Docker resources/cache identify chesi → approved cleanup perform cheyyi.
df -h → free space verify cheyyi.
Pipeline retry chesi → job successful ga run ayyindo check cheyyi.

---

30. GitLab CI/CD lo failed job ni debug chesi final ga ela close chesthav?
Answer:
I start with the failed job log, identify the root cause, reproduce the issue where practical, apply the smallest safe fix, and rerun the affected job or pipeline.
After success, I verify that downstream stages also pass and document the root cause and fix if the issue is operationally significant.
Simple ga gurthupettuko:
Job log → root cause → reproduce → fix → rerun → downstream verify → document.
Eng: I troubleshoot from the job logs, fix the root cause, rerun the pipeline, verify downstream stages, and document the resolution.
Example:
GitLab → CI/CD → Pipelines → failed pipeline → failed job logs open cheyyi.
Error command identify chesi → Runner lo required command/environment test cheyyi.
Configuration/code fix chesi → Retry job.
All stages green ayyaya verify cheyyi.
Recurring/Production-impact issue ayithe → Jira incident/problem ticket lo RCA and resolution update cheyyi.